Studio Nx

JS 24 — initProfileForm()

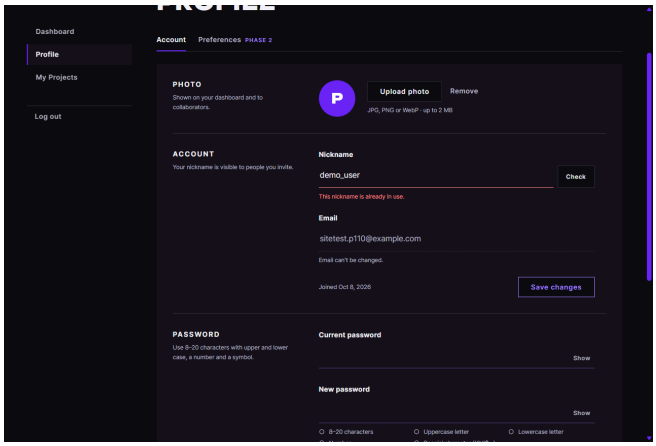
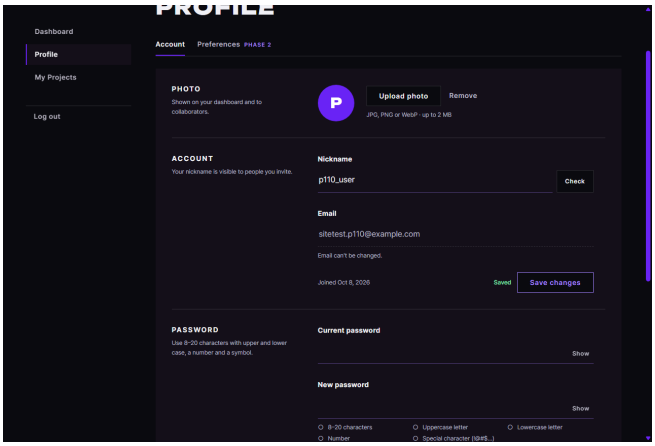
2026-10-07

JS 24 — `initProfileForm()`

항목	내용
파일	`studionx/frontend/static/js/pages/userinfo.js` · `userInfo.html` (Saved 자리) · `static/css/pages/userinfo.css`
작업	P1-10 회원 정보 화면 JS — 닉네임 중복 확인 · 저장 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-08 · 사용자 검토
이전 · 다음	23 `initPhotoUpload()` · 25 `initPasswordForm()`

1. 하는 일

닉네임을 바꿀 때 **가입 화면과 같은 방식**으로 중복을 확인하고, **Save changes** 로 저장한다.

이미 쓰는 닉네임	저장됨
	

입력 (지금 `p110_user`)	칸 아래
`demo_user` (남이 씀)	입력을 멈추고 0.5초 → 빨강 "This nickname is already in use."
`a`	자동 확인 안 함 · Check 누르면 "Use 2–20 characters."
`P110_User` (대소문자만)	**묻지 않고** "✓ Available" — 내 닉네임이라서
`p110_user` (지금 그대로)	아무 글도 없음
`zeta_p110` (새 이름)	"✓ Available"
탈퇴 3일 안인 닉네임	"You can use this nickname after Oct 9."

Save changes	결과
확인을 통과한 값	저장 → 버튼 옆 초록 **Saved** (3초 뒤 사라짐), 사진이 없으면 동그라미 첫 글자도 새로(`Z`)
확인 전에 바로 누름	"Click Check to confirm this nickname is available." — 저장하지 않음
사용 중으로 나온 값	그 문구 그대로, 저장하지 않음
확인 뒤 남이 먼저 가져감 (서버 409)	칸에 "This nickname is already in use."
그 밖의 서버 오류	폼 아래 `#profile-error`

2. 언제 쓰나

userInfo.html 이 열릴 때 한 번, 22 `loadMe` **다음에**. 22 가 닉네임 칸에 `data-checked = 지금 닉네임` 을 넣어 두어서, 이 함수는 그것을 "지금 내 닉네임" 으로 삼는다.

쓰는 API	서버
`GET /api/v1/auth/check-nickname?nickname=` (로그인 없이)	백엔드 24 — 가입 화면과 같은 API
`PATCH /api/v1/users/me`	백엔드 41 → 37

HTML · CSS 에 더한 것: Save changes 버튼 옆에 `<span id="profile-saved" role="status">` 와 감싸는 `<div class="ui-save__end">`, 초록 글자 · 비면 숨김 · 휴대폰에서는 버튼 위 가운데.

3. 코드

import (파일 맨 위 — 22 와 같음, 추가 없음). **`api` · `setFieldState` · `renderAvatar` (22)** 를 쓴다.

상수

```
/** 닉네임 길이 (서버 Nickname 타입 · 가입 화면과 같음) */
const NICKNAME_MIN = 2;
const NICKNAME_MAX = 20;
```

함수 — 준비와 확인

```
export function initProfileForm() {
  const form = document.getElementById('profile-form');
  const input = document.getElementById('profile-nickname');
  const field = document.getElementById('nickname-field');
  const checkButton = document.querySelector('#profile-form [data-check="nickname"]');
  const error = document.getElementById('profile-error');
  const saved = document.getElementById('profile-saved');
  const submit = form ? form.querySelector('button[type="submit"]') : null;
  if (!form || !input || !field) return;

  let current = field.dataset.checked || input.value.trim(); // 지금 내 닉네임 (22 가 넣어 둠)
  let timer = null;
  let savedTimer = null;
  const isValid = (value) => value.length >= NICKNAME_MIN && value.length <= NICKNAME_MAX;

  const check = async () => {
    clearTimeout(timer);
    delete field.dataset.checked; // 이번 확인을 통과해야 다시 남김
    const value = input.value.trim();
    if (!isValid(value)) {
      setFieldState(field, 'error', 'Use 2–20 characters.');
      return;
    }
    if (value === current) {
      field.dataset.checked = value;
      setFieldState(field, null); // 내 닉네임 그대로 — 안내 글로
      return;
    }
```

```
    if (value.toLowerCase() === current.toLowerCase()) {
      field.dataset.checked = value;
      setFieldState(field, 'ok', '✓ Available');
      return;
    }
    setFieldState(field, null, 'Checking…');
    let result;
    try {
      result = await api.get('/auth/check-nickname?nickname=' + encodeURIComponent(value), { auth: false
});
    } catch (err) {
      if (input.value.trim() !== value) return; // 그사이 입력이 바뀜 — 옛 응답은 버림
      setFieldState(field, 'error', err.status === 422 ? 'Use 2–20 characters.' : err.message);
      return;
    }
    if (input.value.trim() !== value) return;
    if (result.available) {
      field.dataset.checked = value;
      setFieldState(field, 'ok', '✓ Available');
    } else if (result.reason === 'REJOIN_TOO_SOON') {
      const day = new Date(result.available_at).toLocaleDateString('en-US', { month: 'short', day: 'nume
ric' });
      setFieldState(field, 'error', `You can use this nickname after ${day}.`);
    } else {
      setFieldState(field, 'error', 'This nickname is already in use.');
    }
  };

  input.addEventListener('input', () => {
    delete field.dataset.checked;
    setFieldState(field, null);
    if (saved) saved.textContent = '';
    clearTimeout(timer);
    if (isValid(input.value.trim())) timer = setTimeout(check, 500);
  });
  if (checkButton) checkButton.addEventListener('click', check);
```

함수 — 저장

```
  form.addEventListener('submit', async (event) => {
    event.preventDefault();
    clearTimeout(timer);
    if (error) error.hidden = true;
    const value = input.value.trim();
    if (!isValid(value)) {
      setFieldState(field, 'error', 'Use 2–20 characters.');
      input.focus();
      return;
    }
    if (field.dataset.checked !== value) {
      if (!field.classList.contains('is-error')) {
        setFieldState(field, 'error', 'Click Check to confirm this nickname is available.');
      }
      input.focus();
      return;
    }
```

```
    if (submit) submit.disabled = true;
    try {
      const me = await api.patch('/users/me', { nickname: value });
      current = me.nickname;
      input.value = me.nickname;
      field.dataset.checked = me.nickname;
      setFieldState(field, null);
      renderAvatar(me); // 사진이 없으면 첫 글자가 바뀔 수 있음
      if (saved) {
        saved.textContent = 'Saved';
        clearTimeout(savedTimer);
        savedTimer = setTimeout(() => { saved.textContent = ''; }, 3000);
      }
    } catch (err) {
      if (err.status === 409 || err.status === 422) {
        delete field.dataset.checked;
        setFieldState(field, 'error', err.status === 422 ? 'Use 2–20 characters.' : err.message);
        input.focus();
      } else if (error) {
        error.textContent = err.message || 'Something went wrong. Please try again.';
        error.hidden = false;
      }
    } finally {
      if (submit) submit.disabled = false;
    }
  });
}
```

최신 문법으로 쓰면 (공부용 — 바뀌는 부분만)

```
const submit = form?.querySelector('button[type="submit"]');
let current = field.dataset.checked ?? input.value.trim();
…
checkButton?.addEventListener('click', check);
error && (error.hidden = true);
…
const { nickname } = await api.patch('/users/me', { nickname: value });
```

최신 문법	실제 코드	뜻
`form?.querySelector(…)`	`form ? form.querySelector(…) : null`	폼이 있을 때만
`a ?? b`	`a || b`	없으면 b (여기선 빈 닉네임이 없어서 `||` 와 결과가 같음)
`checkButton?.addEventListener(…)`	`if (checkButton) checkButton.addEventListener(…)`	있을 때만
`error && (error.hidden = true)`	`if (error) error.hidden = true`	`&&` 로 줄여 쓰는 습관 — 읽기 어려움
`const { nickname } = …`	`const me = …` + `me.nickname`	응답에서 칸 꺼내기

문법 풀이

식	뜻
`let current = …`	저장에 성공하면 바뀌므로 `const` 가 아니라 `let`

식	뜻
`value.toLowerCase() === current.toLowerCase()`	대소문자만 다른지
`setTimeout(check, 500)` · `clearTimeout(timer)`	입력을 멈춘 뒤 0.5초 (가입 15 와 같음)
`encodeURIComponent(value)`	주소에 넣을 수 있게 (`+` · `&` 등)
`delete field.dataset.checked`	`data-checked` 속성 지우기
`form.requestSubmit()` (시험에서)	사람이 Save 를 누른 것처럼 `submit` 을 일으킴

4. 흐름

입력할 때: 확인 표시 · 문구 · Saved 를 지움 → 길이가 맞으면 0.5초 뒤 확인 예약. **확인:** 길이 → 지금 그대로면 끝 → 대소문자만 다르면 ✓ → 서버에 묻기 → 늦게 온 답은 버림 → 사용 가능 · 탈퇴 3일 · 사용 중. **저장:** 길이 → 확인을 통과한 값인지 → 버튼 잠금 → PATCH → 성공이면 Saved · 첫 글자, 409 · 422 는 칸에, 그 밖은 폼 아래 → 잠금 풀기.

5. 왜 이렇게

선택	이유
가입 화면(15 · 17)과 같은 규칙 · 문구 · 0.5초	같은 닉네임 규칙을 두 화면에서 다르게 보여 주면 헷갈린다
대소문자만 바꾸면 묻지 않음	확인 API 는 대소문자를 구분하지 않고 비교해 **내 닉네임을 "사용 중"** 으로 답한다. 서버의 저장(37)은 이것을 허용하므로 화면도 맞춰야 한다
지금 닉네임은 22 가 남긴 `data-checked` 에서	따로 저장하지 않고 22 와 이어 쓴다. 저장에 성공하면 `current` 를 바꾼다
확인을 통과해야 저장	템플릿 요구 · 가입과 같음. 대부분 0.5초 자동 확인이 끝나 있어 따로 누를 일은 적다
409 를 칸에	확인과 저장 사이에 남이 먼저 가져간 경우 — 고칠 곳을 바로 알려 준다
Saved 는 3초 뒤 사라짐 · 입력하면 바로 지움	오래 남으면 "지금 칸의 값이 저장됐다" 로 오해할 수 있다
탈퇴 3일 문구를 "can use … after" 로	가입 화면의 "sign up again after" 는 이 화면에 맞지 않는다 (P1-08 37 과 같은 이유)

6. 시험 (실제 userInfo.html · 실제 서버 · 15개)

시험	확인한 것
처음	지금 닉네임 · `data-checked` · 문구 없음
그대로 Save	Saved, PATCH 1번
`demo_user`	입력하자 Saved · 확인 표시 지움 → 0.3초엔 요청 0번 → 0.5초 지나 "already in use" → Save 해도 PATCH 0번
`a`	자동 확인 0번 → Check → "Use 2–20 characters." (요청 0번)
`P110_User`	요청 0번으로 ✓ → 저장 → 서버에도 `P110_User`
`zeta_p110`	✓ → 저장 → 첫 글자 `Z` → 3초 뒤 Saved 사라짐
확인 전에 바로 Save	"Click Check…", PATCH 0번

시험	확인한 것
서버 409 (확인 표시를 꾸며 남의 닉네임으로 저장)	칸에 "already in use", 확인 표시 지움, 버튼 다시 켜짐
되돌리기	서버에 `p110_user`

15/15 통과. 22(8개) · 23(9개) 시험도 다시 돌려 통과.